

Lecture 4 - System Abstraction & "Growing a Language": Case Study Presentations in Week 5

Topic: System Abstraction, Domain Modeling, and Designing for Growth

Format: Each Group: 5-Minute Live Presentation & Discussion

Lecturer's Opening Hook: The "One-Syllable" Challenge (3 mins)

(Before splitting into groups, start the class by reading this exact script to demonstrate Guy Steele's coreconcept).

Lecturer: "We need to build a new tool for this school. The tool tracks are in the room. A user has a card. The card lets them in. The tool saves time and name. We want the tool to be fast. We want the tool to be safe. Did you notice anything weird about how I just spoke? Every single word I used was exactly **one syllable**. Just like Guy Steele in his famous OOPSLA '98 keynote, I restricted myself to 'primitives'. It sounds childish, right? It takes forever to explain complex ideas this way.

As architects, if we don't design rich, extensible vocabulary (our Domain Model), our developers will be stuck coding in 'one-syllable' primitives forever. Today, we are going to design a system that can *grow*."

The Activity: "Swinburne Smart-Campus Consulting Briefs"

The Core Scenario: Swinburne University wants to replace physical paper attendance sheets with an "Automated Attendance and Access Control System" using swipe cards. Initially, it will track attendance for lectures to reduce manual processing. *However*, the university explicitly stated that at a later stage, the system must grow to handle physical door access to secure facilities, and eventually, library book checkouts.

(Students either form groups or your team of Assignment and select one of the following briefs to present).

Group 1: The Domain Architects (Abstractions)

Scenario: Your job is to define the boundaries of the problem and create the initial abstract map of the university's attendance process before any code is written. **Presentation Prompts:**

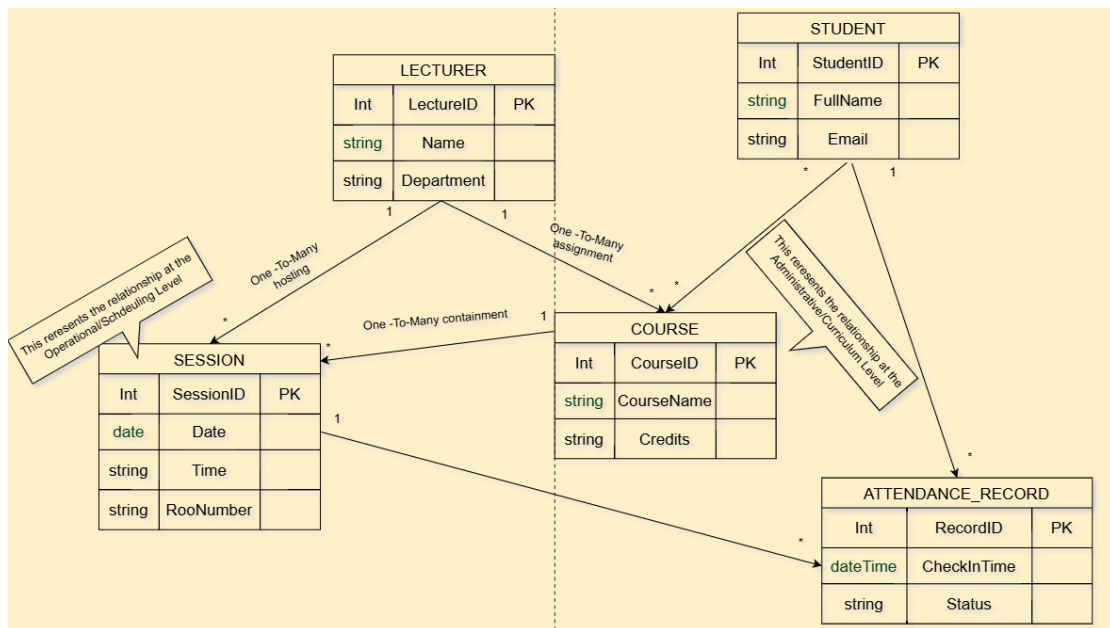
1. **Goals:** Identify the primary business goal and at least two secondary objectives of the system described.

=> The primary business goal is the system should capture, verify, and log student attendance records across all courses to monitor academic compliance and engagement automatically.

=> Two secondary objectives of the system described including:

- Automate Reporting: Eliminate manual paper sheets by instantly providing attendance metrics to academic staff and university administrators.
- Early Intervention: Identify chronically absent students automatically to trigger academic counseling or administrative alerts.

2. **Domain Model:** Draw/Describe the core Domain Model for this system (e.g., what are the entities and relationships?).



3. The Abstraction Link: How is your domain model an *abstraction* of the physical university? Give an example of a messy "real-world" detail regarding classrooms or students that you purposely left out of your model to manage complexity.

=> In detail, the model ignores physical infrastructure and human complexities which reduce the tangible campus into logical, data driven entities (Student, Session) that can be processed by database engines to manage complexity.

=> Example of messy 'real-world'

- **Classroom Logistics:** We purposely left out where a student physically sits, whether they entered through the front or back door, room capacity limits, or the physical condition of the room's air conditioner.
- **Student Behaviors:** We left out why a student is late, what they are wearing, or whether they forgot their physical ID card and had to verbally tell the lecturer. Including these details adds zero value to the digital record and causes unnecessary architectural bloat.

Group 2: The Task Engineers (Implied Workflows)

Scenario: The client brief only explicitly mentions "tracking attendance" and "door access." As an architect, you know there are dozens of *implied* human tasks required to make this actually work in the real world.

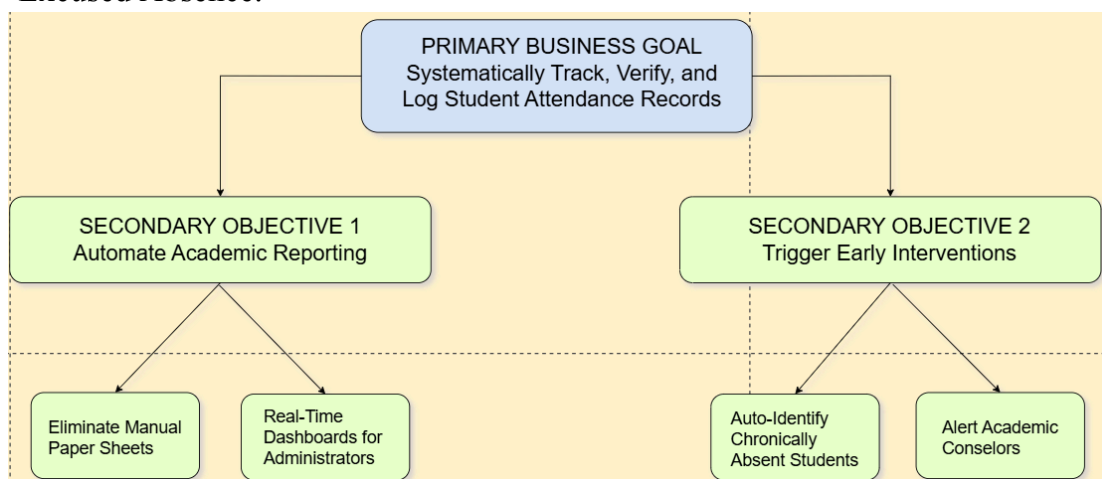
Presentation Prompts:

1. **Implied Tasks:** Identify and briefly describe at least 3 critical "User Tasks" that are NOT mentioned in the brief but are absolutely required for the system to function (Hint: Think about what happens when things go wrong).

=> **Manual Override and Dispute Resolution (Staff Task):** If a student's card is demagnetized, lost, or the reader errors out, a lecturer or administrator must have a workflow to manually override the system and mark the student as "Present" to resolve attendance disputes.

=> **Credential Lifecycle Management (Admin Task):** A system cannot map door access without a workflow for onboarding new student RFID cards/mobile tokens, revoking lost cards instantly for security, and assigning temporary visitor passes for guest lecturers.

=> **Exception Flagging & Leave Management (Student/Staff Task):** Students will inevitably miss class due to legitimate reasons (e.g., medical illness, family emergencies). The system requires a workflow for students to upload digital medical certificates and for staff to review and convert an "Absent" status into an "Excused Absence."



2. **The "Buddy" Problem:** The brief mentions stopping students from signing for friends. With a swipe card system, what is the new physical risk (e.g., "buddy swiping"), and how might you handle it through software or process design?

=> **Software Design Mitigations:**

- **Anti-Passback & Cooldown Logic:** The software can enforce a strict rate-limiting policy per card reader node. If a single device registers multiple card swipes within an impossibly short window (e.g., less than 1.5 seconds apart), the software flags an "Anomaly Detection Alert" for the lecturer's dashboard to audit the line.
- **Timestamp Cross-Referencing:** If the university utilizes a multi-door system, the backend can flag instances where a student's card swipes into Classroom A, but their friend's card (which they allegedly brought) swiped into the campus library or a parking gate at the exact same minute.

=> **Process Design Mitigations**

- **Two-Factor Authentication (2FA) via Mobile Geofencing:** Instead of relying solely on a physical plastic card, the system can require the student to open the university mobile app. The app verifies that the phone's internal GPS is within 5 meters of the classroom coordinates (**Geofencing**) or matching a dynamic BLE (Bluetooth Low Energy) beacon emitted by the lecturer's desk.
- **Randomized Visual Audit:** When a card is swiped at the door, the student's database profile picture can instantly display on a small screen facing the lecturer or a front desk monitor, allowing for a rapid, real-time visual check.

Group 3: The Quality & Risk Consultants

Scenario: You are responsible for ensuring the system survives contact with the real world during the 5-minute rush between classes. **Presentation Prompts:**

1. **Quality Attributes:** Identify 3 specific Non-Functional Requirements (Quality Attributes) that are critical for this specific system.

=> Performance Efficiency (Time Behavior & Throughput): The system must handle high peak throughput. It must process a door swipe or mobile check-in token and return a visual confirmation (success/fail indicator) in less than 1.5 seconds per student to prevent physical bottlenecks and gridlock at classroom doors.

=> Availability & Fault Tolerance: The system must achieve 99.99% operational uptime during high-traffic class transition windows. If the central network dips or a database lock occurs during those critical 5 minutes, the system must gracefully degrade (e.g., caching check-ins locally on the door hardware) rather than crashing and locking students out.

=> Usability (Operability): The check-in interface must require zero cognitive load or complex multi-step navigation from the user. It must be a simple, single-action mechanism (e.g., tap-and-go) that operates successfully even when students are walking quickly past the terminal.

2. **Verification:** Pick one of your quality attributes. How exactly would you verify/test it before the university pays you for the software?

=> Test Environment SetUp:

- **Workload Simulation:** We will use specialized software testing tools (such as Apache JMeter or Locust) to spin up virtual users simulating **2,000 concurrent active students** hitting the attendance API endpoints simultaneously within a concentrated 3-minute window.
- **Hardware Emulation:** We will connect a test cluster of 20 physical card readers/gateways to the staging server to replicate actual physical hardware signal lag.

=> Pass/Fail Criteria:

- **Response Latency:** 95% of all simulated check-in requests must process and return an HTTP **200 OK** status code in **under 1.0 second**.
- **Error Rate:** The system must experience a **0.0% failure/drop rate** under maximum benchmarked peak load. Any dropped requests, timeout errors, or unhandled database deadlock exceptions will constitute a test failure.

3. **Hidden Concerns:** What is one major security, privacy, or environmental concern you have about this system that the university hasn't thought of yet?

=> Security/Privacy:

- Response Latency: 95% of all simulated check-in requests must process and return an HTTP **200 OK** status code in under 1.0 second.
- Error Rate: The system must experience a 0.0% failure/drop rate under maximum benchmarked peak load. Any dropped requests, timeout errors, or unhandled database deadlock exceptions will constitute a test failure.

=> Environmental/Hardware

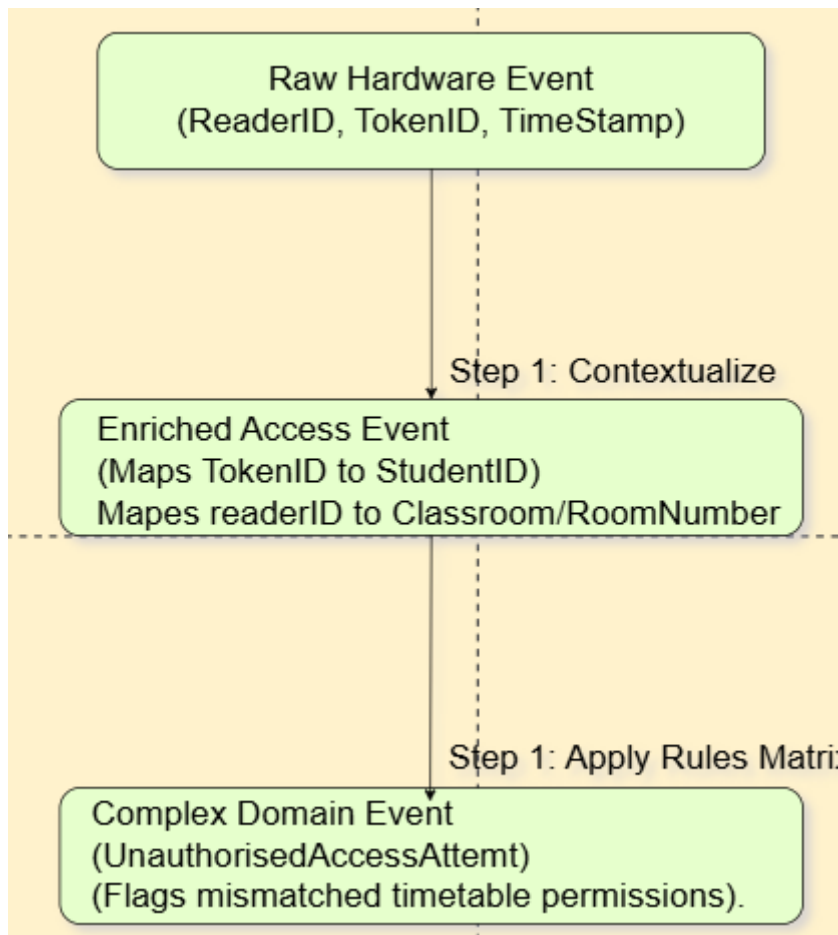
- The Concern: Deploying IoT smart card-readers or biometric scanners outside *every single classroom door* across an entire campus creates an immediate physical maintenance debt.
- The Risk: These edge devices run 24/7, are prone to physical wear-and-tear from rushing students, and contain toxic heavy metals. Over a 3-to-5-year lifecycle, the university faces a massive wave of hardware failures, component degradation, and inevitable electronic waste (e-waste) management issues that contradict campus sustainability goals.

Group 4: The Extensibility Architects (Applying Guy Steele)

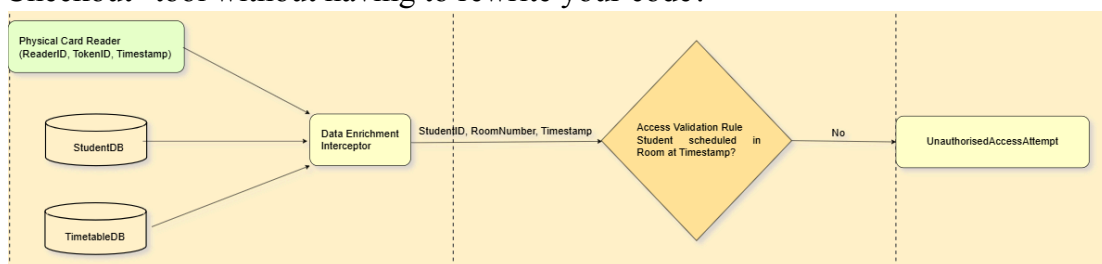
Scenario: In his paper "*Growing a Language*", Guy Steele argues you shouldn't design a massive system all at once, nor a tiny one that can't adapt. You must design a system that acts as a "pattern for growth."

Presentation Prompts:

1. **Primitives to Complex:** In Steele's paper, he starts with 1-syllable primitives to build complex words. In our system, what is the lowest-level data "primitive" generated by the hardware? How do you combine those primitives to create a complex domain concept like an `UnauthorisedAccessAttempt`?



2. **"Going Meta"**: Steele states: *"We should now think of a language design as a tool for making more tools."* How do you design the core of this attendance system so that next year, the Library IT team can use your core to build a "Book Checkout" tool without having to rewrite your code?



Group 5: The System Philosophers (Cathedrals vs. Bazaars)

Scenario: Swinburne management is debating how to build this system. The central IT department wants to build a "Cathedral" (a massive, rigid, top-down master plan for the whole university). The student developer club wants to build a "Bazaar" (open APIs where any department can hack together what they need).

Presentation Prompts:

1. **The Cathedral Risk:** According to Steele and Alexander (quoted in the paper), what is the primary danger of Swinburne imposing a strict "Master Plan / Cathedral" architecture on this system from Day 1?

=> **The Core Danger:** A master plan assumes that the architects can predict every single future requirement, human behavior, and operational edge case before a single line of code is written. In a dynamic university ecosystem, this is impossible.

=> **The Inevitable Outcome:** By building a massive, rigid, top-down structure, the university accidentally freezes its assumptions into stone. When real-world needs change—such as a sudden policy shift in how attendance affects exam eligibility, or a new requirement to integrate biometric sensors—the "Cathedral" architecture cannot adapt. It becomes too heavy and fragile to modify, leading to massive technical debt, system rejection by users, and eventual project failure.

=> **The Architectural Insight:** As Alexander noted, cities and complex structures grow organically. Trying to dictate the entire lifecycle from the top down destroys the flexibility needed for the system to survive contact with reality.

2. **The Shopping Mall Solution:** Steele ultimately suggests a "Shopping Mall" approach (a compromise between the Cathedral and Bazaar). What does a "Shopping Mall" software architecture look like for a university campus? Who builds what?

=> Central IT Department:

- **The Structural Foundation:** The core database engines, global authentication/identity management servers (OAuth/SSO), and secure data pipelines.
- **The Building Rules (API Gateway):** They define strict, standard, well-documented API contracts, access permissions, and security protocols. This acts as the "mall lease agreement"—if you want to open a shop, you must follow these plumbing and electrical standards.

=> Student Clubs & Individual Departments

- **The Student Club:** Can quickly "hack together" a modern mobile attendance app with beautiful animations for students to check into lectures.
- **The Library Team:** Can use the exact same identity APIs to build their custom book-loan terminal.
- **The Facilities Team:** Can build a dynamic room-booking or smart-lighting tool that activates based on active attendance session timestamps.